

How My Pal JSON works

Deep analysis generated 2026-10-06 · app booted via docker; screenshots are real captures · model claude-sonnet-5

Overview

My Pal JSON is a single-page application served by a small Flask backend (`web/app.py`) and a vanilla-JS frontend with no framework or build step (`web/static/scripts/*.js`, loaded as ES modules). The app has two main "views" toggled by tabs: a Request view for building and sending one HTTP call at a time, and a Chain view for stringing several requests together with data flowing from one response into the next request's variables. Both views share a Collections drawer that pulls and pushes Postman v2.1 `.postman_collection.json` files (and the app's own `.mypaljson_chain.json` chain files) directly from an Azure DevOps or GitHub repository, so collections built here are the same files a teammate sees in Postman.

The target audience is developers and testers who want a Postman-like workflow but with collections under real version control, and who frequently need to generate strongly-typed model classes from sample JSON responses (C#, Python, JavaScript, C++, Java, Go). It's also aimed at people working behind corporate TLS inspection or VPNs, since it runs entirely on the user's machine and can trust a corporate root certificate or reach `localhost`-only services depending on how it's launched (container vs. from source).

End to end, a user picks an HTTP method and types a URL (with optional `{{variable}}` placeholders), fills in headers (one of which can be flagged "IsAuth" so its value is sent as `Bearer <value>` and treated as a secret), types or drops a JSON body, and clicks Send. The browser posts the fully-resolved request to Flask's `/proxy_request` endpoint, which performs the actual HTTP call server-side (so CORS and TLS issues are avoided) and returns a JSON envelope with status, headers, timing and body, which the UI renders under the Response tab. The same JSON can be pretty-printed, turned into a clickable tree (via `/analyze`), or turned into generated class source (via `/model`).

Repository integration lets a user paste a repo URL, authenticate (Personal Access Token, Microsoft browser sign-in, Azure CLI session, or Git Credential Manager), and then Pull collection/chain files into a browsable tree, edit requests, Save them back into the in-memory collection, and Push a commit. Before pushing, the app scans for secret-looking values (auth headers, API keys, tokens) and automatically rewrites them into `{{placeholders}}`, moving the literal values into the user's local-only environment so the repo never stores a real secret. The Chain view works the same way but for ordered sequences of requests, letting a user click a value in one response to capture it as a named output usable as `{{name}}` in any later step.

Architecture

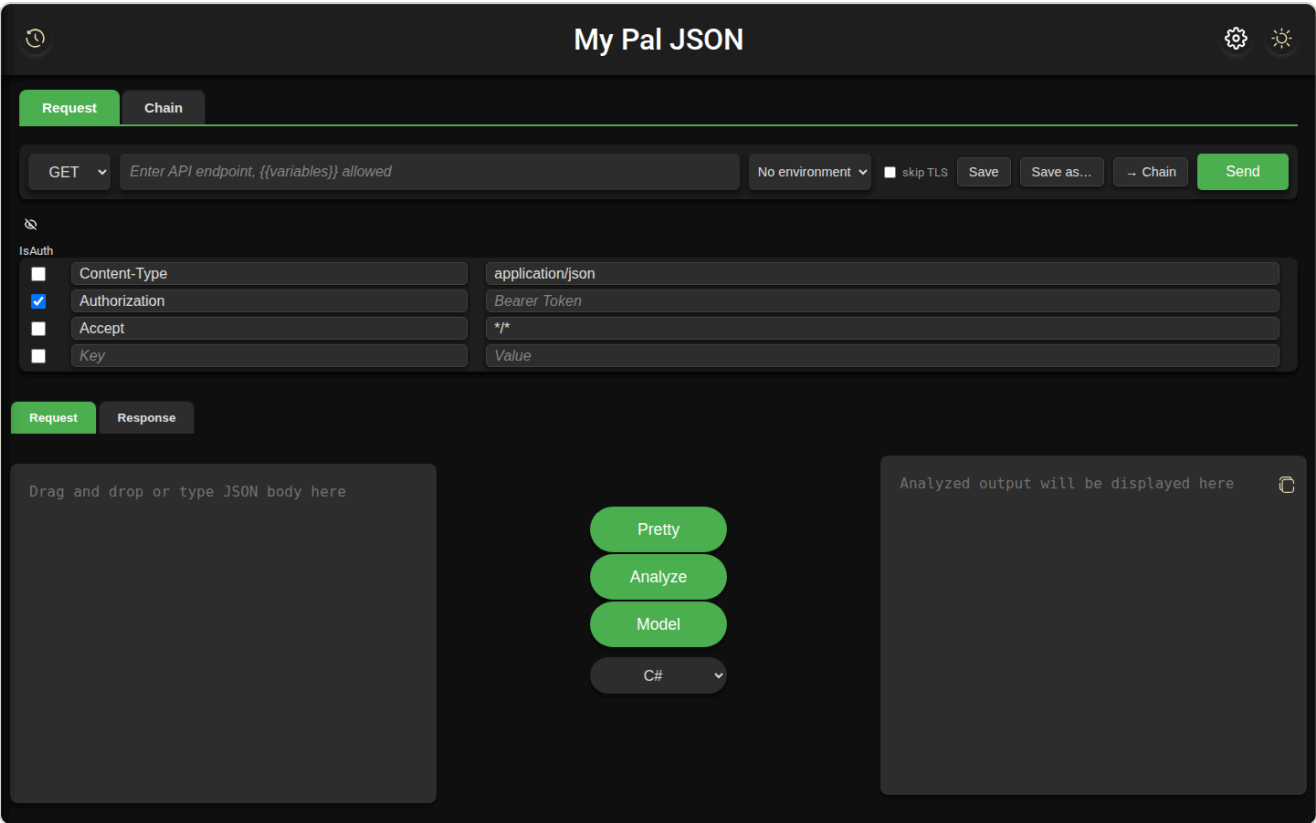
The backend is Flask, organized as blueprints registered in `web/app.py`: `analyze_bp` (`/analyze`, builds an `anytree` representation of JSON structure), `model_bp` (`/model`, dispatches to per-language generators under `web/generators/`), `proxy_bp` (`/proxy_request`, the outbound HTTP proxy with `localhost`→`host-alias` rewriting for container use and TLS skip support), and `repo_bp` (`/repo/*`, file-level git-host access). `web/repo.py` parses a pasted repo URL into a `RepoRef` and picks a provider; `web/providers/azure_devops.py` and `web/providers/github.py` implement a shared `BaseProvider` interface (ping, list_branches, list_dir, get_file, put_file, delete_file) against the Azure DevOps Git REST API and GitHub Contents/Trees API respectively, with optimistic-concurrency version checks that surface as "File changed on the server, pull again". `web/auth_sources.py` supplies tokens from four sources — a PAT sent from the browser, the Azure CLI (`az account get-access-token`), Git Credential Manager (`git credential fill`), or an MSAL browser auth-code+PKCE flow whose redirect is handled by the `/` route in `app.py` itself.

The frontend is plain ES modules under `web/static/scripts/`, each owning one concern: `send-request.js` (request state/send), `headers-management.js` (the headers grid), `environments.js` and `db.js` (IndexedDB-backed environments/settings/repos/chain-drafts), `collections-store.js`/`collection-browser.js`/`postman.js` (pulling, parsing, and patching Postman v2.1 JSON while preserving unknown fields), `secret-scrub.js` (the placeholder-allocation logic

used before every push), and the `chain/` subfolder (`chain-store.js`, `chain-runner.js`, `chain-view.js`) implementing chain drafts, sequential execution with variable resolution, and the step-by-step UI. `json-tree-view.js` renders clickable JSON trees used both by Analyze output and by the chain's "pick from response" feature. There is no bundler; `json-analyzer.html` loads these scripts as native `

Page: /

The single page of the application — a request builder/sender with JSON tooling (pretty-print, structure analysis, model-class generation) and a Chain builder, backed by a Collections drawer for pulling/pushing Postman-format files from a git repository.



Controls & fields

CONTROL / FIELD	WHAT IT DOES
Collections icon button (collectionsBtn)	toggles the Collections drawer open/closed; <code>`openDrawer()``/``closeDrawer()``/``toggleDrawer()``</code> in <code>collection-browser.js</code> , auto-pulls on first open if a repo is active
Settings icon button (settingsBtn)	opens the Settings modal on the Repositories pane; <code>`openSettings('repos')``</code> in <code>settings-panel.js</code>
Theme toggle button (themeToggleBtn)	flips <code>`data-theme``</code> between light/dark on <code>`<html>``</code> and persists the choice to <code>`localStorage``</code> ; <code>theme-toggle.js</code>
Request tab	switches the main view to the Request builder; <code>view-switch.js</code> <code>`showView('request')``</code>

(button: Request / #viewRequestTab)	
Chain tab (button: Chain / #viewChainTab)	switches the main view to the Chain builder/list; <code>view-switch.js`showView('chain')`</code>
HTTP method select (GET/POST/PUT/PATCH/DELETE/HEAD/OPTIONS)	sets the method used when sending, saving, or adding this request to a chain; read by <code>`dataStore.currentRequest()`</code> in <code>send-request.js</code>
API endpoint input ("Enter API endpoint, {{variables}} allowed")	the request URL, may contain <code>`{{var}}`</code> placeholders resolved against the active environment/chain scope by <code>`resolveRequest`</code> in <code>variables.js</code> ; unresolved names are flagged in the status line
Environment select (envSelect, "No environment" when none exist)	picks the active environment used to fill <code>`{{variables}}`</code> in URL/headers/body; <code>environments.js`setActiveEnvId`</code>
Skip TLS checkbox (unlabeled input[checkbox] next to Send, "skip TLS")	when checked, passes <code>`verifyTls:false`</code> to <code>`/proxy_request`</code> , disabling certificate verification for self-signed local hosts; <code>proxy.py</code> disables the <code>urllib3</code> insecure-request warning accordingly
Save button (saveRequestBtn)	writes the current form back into the loaded collection request (also bound to Ctrl+S); uses <code>`postman.js`requestToItem`</code> and marks the collection file dirty in <code>collections-store.js</code>
Save as... button (saveAsBtn)	saves the current form as a new request in a chosen collection/folder via the collection browser's save-as flow
→ Chain button	adds the current request as a new step in the currently open chain draft; <code>`newStep()`</code> in <code>chain-store.js</code>

(sendToChaiBtn)	
Send button (sendBtn)	resolves variables, posts the request to Flask's <code>`/proxy_request`</code> , and renders the returned envelope under the Response tab; <code>`sendRequest()`</code> in <code>send-request.js</code> , handled server-side by <code>`proxy_request()`</code> in <code>proxy.py</code>
Toggle headers button (toggleHeadersBtn, eye/eye-slash icon)	shows or hides the headers grid and its IsAuth column label; <code>headers-toggle.js</code>
Auth checkbox per header row (title: 'Auth header: sent as "Bearer <value>" and never written to the repo as a literal')	marks a header as the auth header so its value is prefixed with <code>`Bearer `</code> when sent (<code>`collectHeaders`</code> in <code>send-request.js</code>) and is scrubbed into an environment <code>`{{placeholder}}`</code> before any repo push (<code>secret-scrub.js</code>)
Header Key input	the header name typed into the grid; empty rows are pruned and a trailing blank row auto-added by <code>headers-management.js</code>
Header Value input (placeholder "Bearer Token" when key is Authorization, else "Value")	the header's value; combined with the key by <code>`readHeaders()`</code> into the request sent to the proxy
Request tab / Response tab (sub-tabs above the body textarea)	switch the single JSON textarea between showing the outgoing request body and the last response body; <code>tab-switch.js</code>
Request body textarea (inputJson, "Drag and drop or	holds the JSON (or raw text) request body; accepts typed text or a dropped <code>` .json `</code> /text file (<code>input-drop.js</code> reads the file and fires an input event); captured into <code>`dataStore.requestData`</code> while the Request sub-tab is active

type JSON body here")	
Pretty button (prettyBtn)	sanitizes and `JSON.parse`/`JSON.stringify`'s the body textarea with 4-space indentation into the output box, or shows an "Invalid JSON format" message with the attempted sanitization; make-pretty.js
Analyze button (analyzeBtn)	POSTs the parsed body JSON to Flask's `/analyze`, which recursively builds an `anytree` tree (`build_tree` in analyze.py) and returns a rendered tree string shown in the output box
Model button (modelBtn)	POSTs the parsed body JSON to `/model?language=<lang>`; `model.py`'s `generate_classes` picks a root class name and dispatches to the matching generator in `web/generators/` (csharp, python, javascript, cplusplus, java, go), returning generated class source shown in the output box
Language select (languageSelect: C# / Python / Javascript / C++ / Java / Go)	chooses which generator `/model` uses for the Model button
Output textarea (outputBox, readonly, "Analyzed output will be displayed here")	displays Pretty/Analyze/Model results; read-only, feeds the Copy button
Copy button (copyBtn)	copies the output box's text to the clipboard via the Clipboard API, flashing green on success or red on failure/empty content; copy-output.js
Repository select ("No repositories - open Settings" when empty)	lists saved repos and sets the active one for the Collections drawer; `store.setActiveRepo()` in collections-store.js, triggers `renderRepoControls()` in collection-browser.js
Branch select (branchSelect, "branch" placeholder)	lists branches for the active repo (populated by `repoApi.ping`) and sets the working branch; `store.setBranch()` clears cached files and re-pulls

Pull button (pullBtn)	fetches the repo root and one folder down for <code>`*.postman_collection.json`</code> and <code>`*.mypaljson_chain.json`</code> files and loads them into the tree; <code>`doPull()`</code> → <code>`store.pull()`</code> in <code>collections-store.js</code> , backed by <code>`/repo/tree`</code> and <code>`/repo/file`</code>
Push button (pushBtn)	commits every dirty collection/chain file back to the repo (one commit per file, with secret-scrubbing confirmation first); disabled when there are no dirty files, enabled count shown as a badge
+ button	creates a new, empty collection file in the tree (prompts for a name); <code>`store.createCollectionFile()`</code> in <code>collections-store.js</code>
⚙ / Open Settings controls (gear icon settingsBtn, and "Open Settings" button inside the empty-tree state)	open the Settings modal so the user can add/edit repositories, tokens, and environments; <code>`openSettings('repos')`</code> in <code>settings-panel.js</code>